

THE UNIVERSITY OF SYDNEY

Faculty of Engineering

Study Guidance

A local, policy-aware AI assistance recommender for university tasks

ENGG2112 Project Final Report

Jiachen Lu Siyuan Chen Xin Chen

Semester 2, 2026

Public code repository: https://github.com/jiachen28/study-guidance

Executive Summary

Students increasingly use general-purpose AI tools for coursework, yet the most suitable form of assistance depends on the task and the assessment rules. We built Study Guidance, a local web application that routes a natural-language task into Coding, Research, Workflow, or None and returns a suggested tool type, explanation, human checks, restrictions and source links. The application combines a locally hosted Qwen2.5 3B model, a small attributed reference index and deterministic safety checks. A keyword-based classifier serves as a comparison baseline and fallback. The team authored 60 scenarios, independently labelled each scenario three times, and reserved 20 scenarios for evaluation. Initial unanimous agreement was 59/60 (98.3%; Fleiss' $\kappa = 0.985$). On the held-out set, the language model and the integrated system each classified 15/20 cases correctly (75%, macro-F1 0.703), compared with 13/20 (65%, macro-F1 0.640) for the keyword baseline. The model missed four of five Coding examples, showing a significant practical weakness despite its overall improvement. The findings support a limited proof of concept, not deployment-ready advice. Human verification and explicit assessment-rule checks remain essential.

Contents

1 Problem and objectives **3**

2 Background and project scope **3**

3 System design and implementation **3**

3.1 Architecture 3

3.2 Categories and safeguards 4

3.3 Reproducibility and testing 4

4 Study method **4**

4.1 Dataset and annotation protocol 4

4.2 Comparison and measures 5

5 Results **5**

5.1 Class-specific performance and error analysis 6

6 Quality, limitations and interpretation **7**

7 End-to-end use and socio-technical implications **7**

8 Recommendations and conclusion **8**

A Reproduction commands and evidence index **8**

1 Problem and objectives

A student asking for help with a failing test suite needs a different workflow from one checking the provenance of a chart or coordinating a group presentation. A generic chatbot can answer all three, but its confidence does not establish whether the tool is appropriate, whether its sources are reliable or whether an assessment permits AI assistance. The project therefore asks: *Can a small local system recommend an appropriate type of AI assistance for common university tasks while making the required human checks visible?*

We set four objectives: (1) accept free-form task descriptions through a usable local interface; (2) route requests into four actionable categories; (3) attach practical verification and safety guidance, including refusal or redirection for explicitly prohibited tasks; and (4) evaluate the routing against independently annotated scenarios and a reproducible baseline. The output is advice about an appropriate tool *type*, not an endorsement of a commercial service or a determination of university policy.

2 Background and project scope

The University of Sydney's responsible AI guidance requires students to check the rules applying to their particular assessment and to use AI responsibly [1]. UNESCO emphasises human agency and educational oversight [2]; the NIST AI Risk Management Framework provides a broader basis for considering risks and affected stakeholders [3]. OWASP's guidance on large language model applications highlights the risks of treating generated or retrieved text as trusted instructions [4]. These sources informed the application's safeguards, but their general guidance is not a substitute for the exact instructions of an individual unit.

Unlike a general-purpose chatbot, a keyword router is easy to inspect but fails when task wording is indirect; a language model can interpret intent but may confidently invent restrictions. Retrieval-augmented prompting can add contextual references, while explicit policy rules provide deterministic handling of selected prohibited requests. We therefore combined these complementary mechanisms rather than relying on any single one. The original proposal called for a student-facing guidance tool and a measurable comparison with a simple baseline. The completed product narrows the scope to local inference, an authored scenario dataset and a limited reference index. We did not collect student usage logs, survey participants or claim representative population performance. The intended audience is a student seeking an initial route for a task, followed by a human decision about whether and how to use a tool.

3 System design and implementation

3.1 Architecture

The application uses Python's standard library, a Flask-free local HTTP service, a browser interface and Ollama running `qwen2.5:3b-instruct`. It binds to `127.0.0.1:8765`. The student enters a task, which passes through rule-based safety assessment and TF–IDF retrieval over 16 short, attributed reference notes. A structured prompt asks the local model for a category, explanation, verification step and limitations. A final policy layer selects the displayed recommendation and attaches tool-type suggestions and source links. The reference index uses lowercase word tokenisation, smoothed inverse document frequency, log-scaled term frequency and cosine similarity; only positively matching notes are returned, with at most four inserted into the prompt. The model receives these notes as context, not as authoritative instructions. Its response is constrained to JSON with four required fields, temperature zero and a bounded generation length. No model weights were trained or fine-tuned: the engineering contribution is a retrieval-and-rule-assisted

inference pipeline. We chose a small locally runnable model to avoid hosted API costs and reduce transmission of task text, accepting the trade-off of limited capacity and device requirements. The local inference design avoids sending the entered task to a hosted model API; it does not make the user's device or inputs automatically secure.

3.2 Categories and safeguards

Coding covers programming, debugging and testing; **Research** covers finding and assessing evidence; **Workflow** covers planning, coordination and routine communication; **None** covers cases where AI assistance is unsuitable or explicitly restricted. The system recommends a programming assistant, academic research tools, planning tools or a non-AI alternative respectively. It adds checklists such as running code tests, verifying primary sources, checking schedules and owners, or consulting assessment rules.

The deterministic policy layer checks selected textual patterns for prohibited assessments, fabricated academic evidence, exposure of private records and unsafe emergency requests. The decision order matters: explicit policy restrictions override the model to None; otherwise a valid model label is used; if the model fails, the keyword baseline supplies a visibly marked fallback. The rules are applied to the task before model inference, so a policy override can affect the final output without changing the raw model label. The local model's answer is treated as advisory. If inference fails, the keyword baseline can provide an explicitly identified fallback. These safeguards are finite pattern checks; indirect or novel unsafe requests may escape them. The retrieval notes are project-authored summaries with attributed URLs, not verified live policy decisions.

3.3 Reproducibility and testing

The repository separates application code (`app/`), scenarios and annotations (`data/`), automated tests (`tests/`) and study outputs (`results/`). `verify.py` runs the test suite, checks the HTTP health endpoint and tests local model inference. The recorded run passed 40 automated tests, the health check returned HTTP 200, and the local model returned a valid result. The evaluation script stores predictions with model identifier, timestamp and a fingerprint of the code and data; rerunning without `-fresh` retains compatible observations. This prevents accidental replacement of the recorded held-out run. The 40 automated tests exercise parsing, policy handling, HTTP endpoints, classification metrics and evaluation workflows; they establish software behaviour on fixtures, not real-world advice reliability. The workflow uses a fixed authored split and a single saved held-out inference run, rather than repeated tuning on the 20 evaluation examples.

4 Study method

4.1 Dataset and annotation protocol

The team wrote 60 short university-task scenarios spanning the four categories, including software testing, evidence checking, group coordination, restricted assessments and safety-sensitive requests. Forty were designated development examples and 20 were held out for final evaluation. These are deliberately authored examples, not a random sample of real students. The held-out set contains five reference examples per category.

Jiachen Lu, Siyuan Chen and Xin Chen each supplied a separate CSV with one independent category label for each of the 60 scenarios (180 labels total). The validation script checks unique IDs, unchanged task descriptions, allowed labels and the 40/20 split. We counted exact three-way agreement and calculated Fleiss' kappa for nominal labels. Fifty-nine cases were unanimous. For S39 (requesting a template to ask teammates to review slides), two annotators selected Workflow

and one selected Research; the stored reference label is Workflow by a recorded two-of-three majority. The repository retains the individual votes and rationale. This majority record should not be confused with evidence that all three participants explicitly endorsed the final decision.

4.2 Comparison and measures

The keyword baseline counts matched phrases for Coding, Research and Workflow, and defaults to None when no keyword is found. The local model predicts from the task and retrieved notes; the final system can override or redirect a prediction through safety rules. We compare all three against the held-out reference labels. Accuracy is the fraction of correct labels. Macro-F1 averages the four per-class F1 scores, giving each category equal weight. Coverage measures the fraction of valid predictions; missing predictions count as incorrect. We also report None precision and recall because false "no AI" recommendations can wrongly discourage legitimate assistance. The four classes are equally represented in the held-out reference labels (five each), so the reported accuracy is a balanced-scenario score, not an estimate under real student task frequencies. We inspect errors against the original saved response text to distinguish category mistakes from unsupported guidance. This is an assessment of *routing*, not a measurement of the factual accuracy of every generated sentence.

5 Results

Table 1: Classification results on the 20 held-out scenarios.

Method	Correct	Accuracy	Macro-F1	None precision	None recall
Keyword baseline	13/20	65.0%	0.640	41.7%	100%
Local Qwen2.5 3B	15/20	75.0%	0.703	83.3%	100%
Integrated system	15/20	75.0%	0.703	83.3%	100%

The model improved accuracy by two cases (10 percentage points) and macro-F1 by 0.063 over the keyword baseline. All 20 predictions were valid, giving 100% coverage. The model and final system produced the same labels for these 20 cases; no measured classification gain can be attributed to the policy layer on this test set. That layer still serves a separate safety and redirection function, tested through targeted software tests.

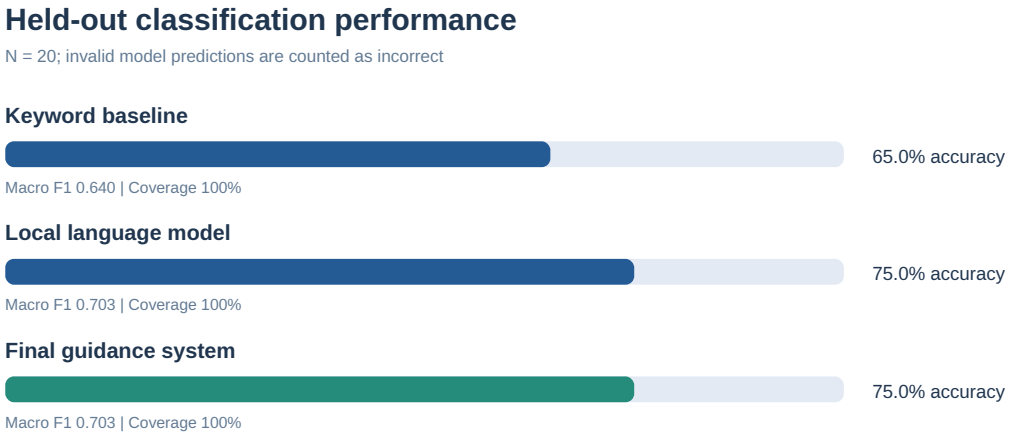


Figure 1: Measured accuracy, macro-F1 and output coverage for the three methods.

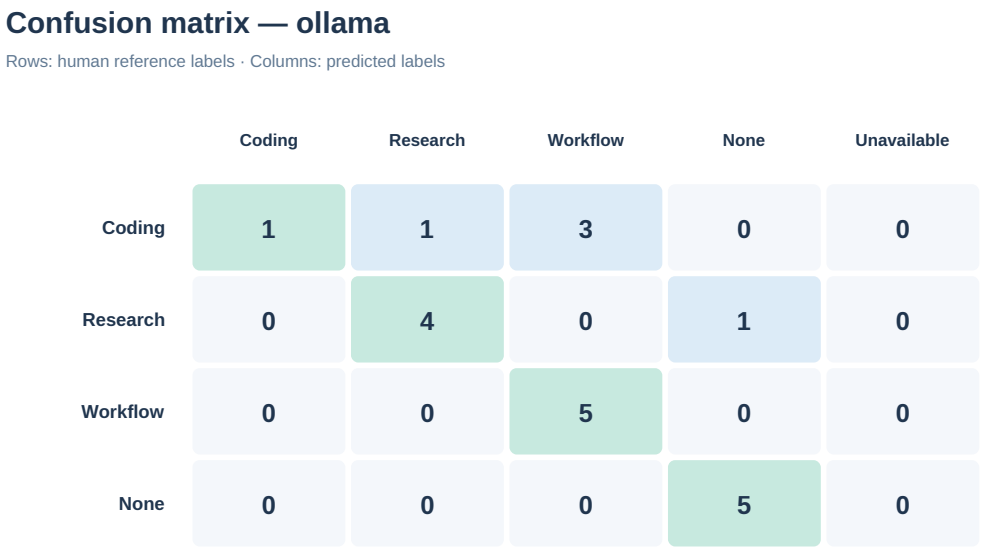


Figure 2: Confusion matrix for the local model. Rows are reference labels and columns are predicted labels.

5.1 Class-specific performance and error analysis

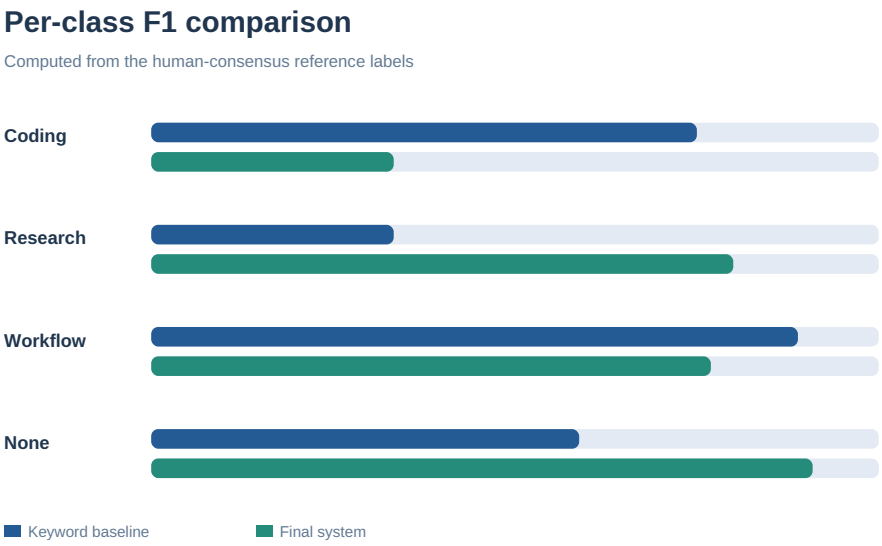


Figure 3: Per-class F1 of the keyword baseline and final system.

The overall result hides an important weakness: Coding recall was only 1/5 (20%), whereas Research recall was 4/5 (80%), Workflow 5/5 and None 5/5. Four of the five model errors involved Coding. In S41, a Django dependency/debugging task was classified as Workflow; in S45, writing malformed-filename security tests was also labelled Workflow; in S53, resolving source-file merge conflicts was labelled Workflow; and in S57, developing C edge-case tests was labelled Research. In S54, researching image authorship and licensing was classified as None rather than Research.

These cases suggest that the model sometimes follows procedural words (“guide”, “tests”, “steps”) instead of the primary technical outcome. S45 is particularly concerning because the generated limitations text incorrectly asserts that AI assistance for security tests is prohibited, although the

scenario itself states no such rule. S54's explanation identifies research activities but still refuses the Research category. These observations come from the saved raw responses and are not inferred from category labels alone. The model's stronger None precision than the baseline reduces, but does not eliminate, inappropriate discouragement of permissible assistance. For the five Coding reference cases, the model recovered only one, making Coding recall (20%) the most consequential measured weakness. A deployment gate should therefore test per-class recall and unsafe-advice rates, not just aggregate accuracy.

6 Quality, limitations and interpretation

The recorded annotation agreement was 59/60 (98.3%) and Fleiss' $\kappa = 0.985$. High agreement supports consistency of this small group's label scheme, but cannot establish that the categories capture every student's intent. The held-out sample has only 20 authored tasks, five per category, and was written by the project team. Results should therefore be read as descriptive performance on this fixed exercise. The 10-point accuracy difference corresponds to only two examples; we do not claim statistical significance or generalisation to the student population.

Classification scores also do not prove that the generated explanations, checks and limitations are reliable. Inspection of saved outputs reveals useful recommendations alongside generic checks and unsupported claims: S42 suggests instructor review without a precise dataset-checking procedure; S45 invents a restriction on security testing; and S60 correctly identifies an emergency as unsuitable for chatbot use but frames its verification around assessment policy rather than immediate emergency response. These are concrete qualitative observations, not a substitute for a completed independent two-reviewer quality study. The repository contains review worksheets, but no independent human-review rate is claimed here without verified reviewer records.

The proposed independent quality rubric separates three dimensions: whether the explanation is supported by the task, whether verification is actionable, and whether limitations are appropriate. Each can be scored 0/1 per saved answer by independent reviewers, with disagreements reconciled against the text. The packaged review sheets are aids for this process; without evidence of independent completed reviews, we do not report a numerical human-quality pass rate. The project has additional constraints. The 16 reference notes are manually authored and can become outdated. The safety rules are pattern-based and may miss paraphrases. Local deployment reduces reliance on a cloud API but does not remove device security, model bias or the possibility of inappropriate user input. Model behaviour may change with model versions, prompts or retrieved notes. A larger independently sourced dataset, a blind annotation process and more systematic adversarial testing are needed before using the tool as a high-confidence adviser.

7 End-to-end use and socio-technical implications

In a practical workflow, a student would describe the task, read the proposed category and suggested tool type, inspect linked sources and complete the verification checklist. For code, this means reviewing changes and running tests; for research, opening original sources and validating claims; for workflow, confirming people, deadlines and consent. If a task appears restricted, the student must consult the actual unit rules or responsible educator. The app should record no sensitive details by default and should not make academic-integrity decisions on behalf of staff.

The stakeholder responsibilities are different. Students must check assessment rules and verify outputs; teaching staff must clarify allowed assistance and handle disputes; maintainers must version policy notes, regression-test overrides and document model changes; the university would need to assess privacy, accessibility and equity before endorsing deployment. Wider deployment would require a maintained policy source, clear ownership of updates, user testing with diverse students, accessible interface design, privacy assessment and a mechanism to contest incorrect

recommendations. Students with limited hardware may not be able to run a local model, creating an access barrier. False None labels could discourage legitimate support; false permissive labels could encourage breaches of assessment rules. These risks make human oversight and explicit uncertainty more important than presenting a single category as authoritative.

8 Recommendations and conclusion

The next technical priority is improving recognition of software debugging, testing and version-control tasks, preferably using new development examples rather than tuning to the held-out mistakes. Specifically, add a separate development set of debugging, security-testing and Git examples; test whether prompts that prioritise the requested deliverable improve Coding recall; then freeze the revision and assess it on a new unseen test set. For S54-like false None decisions, show a clarification request when the restriction is ambiguous instead of confidently refusing. For S45-like unsupported restrictions, require a traceable source or an explicit uncertainty statement before making policy claims. For S60-like emergencies, use a deterministic high-priority response directing users to appropriate immediate assistance, and test paraphrased emergency inputs. Subsequent evaluation should use unseen scenarios written by people outside the team, measure calibration and robustness, and separately assess whether explanations and verification instructions are supported and actionable. The release criteria should be agreed with educators in advance, including minimum Coding recall, a low rate of unsupported prohibitions and reliable handling of emergencies; the present 20-case study cannot validate those thresholds. Policy restrictions should be tested against paraphrases and updated as university rules change.

Study Guidance meets its limited proof-of-concept goals: a functioning local interface, four-way routing, source-linked safety guidance, a reproducible comparison and clear evidence of both improvement and failure modes. The measured 75% held-out accuracy is encouraging relative to the 65% keyword baseline, but the Coding failures and small authored dataset rule out claims of reliable deployment. The most defensible contribution is an inspectable workflow that makes the decision to use AI conditional on task type, evidence and human judgement.

References

[1] University of Sydney, "Using artificial intelligence tools responsibly," https://www.sydney.edu.au/students/responsible-ai-use.html.

[2] UNESCO, "Guidance for generative AI in education and research," https://www.unesco.org/en/articles/guidance-generative-ai-education-and-research.

[3] National Institute of Standards and Technology, "AI Risk Management Framework resources," https://airc.nist.gov/airmf-resources/.

[4] OWASP, "Top 10 for Large Language Model Applications," https://genai.owasp.org/llm-top-10/.

A Reproduction commands and evidence index

```
ollama pull qwen2.5:3b-instruct
python -m app.server
python -m unittest discover -s tests -v
python verify.py
python run_study.py
```

The source of record is the public repository linked on the title page. The main evidence files are `data/scenarios.csv`, the three `data/annotations_*.csv` files, `results/adjudication.csv`, `results/consensus.csv`, `results/evaluation_raw.csv`,

`results/metrics.json`, `results/case_analysis.csv` and the generated figures. Raw model answers are retained for checking individual interpretations. This appendix documents the study outputs; any separate course-required AI-use statement may be added by the submitting team.